



20-CI-PIPELINE-SPEC.md — The 22 gates that stand between a commit and a wedding



Every rule in this pack is only as real as the job that enforces it. This file is the machine that turns the other eleven documents into something Claude Code cannot talk its way past.

1. Pipeline stages

PUSH	---	[1]	Fast gates	~4 min	every commit, blocking
	---	[2]	Correctness	~15 min	every PR, blocking
	---	[3]	Quality	~40 min	every PR touching mode
ls/shaders/colour					
	---	[4]	Platform	~35 min	every PR, matrix Win/m
acOS/Linux					
	---	[5]	Nightly	~4 h	chaos, fuzz, full wedd
ing, benchmarks					
	---	[6]	Release	~1 h	tag only: sign, notari
se, symbols, bundle					

Rule: **stage 1 must stay under 5 minutes**. A slow fast-gate is a gate people learn to bypass.

2. The 22 gates

#	Gate	Stage	Fails when	Source doc
G-01	<code>cargo fmt --check, prettier --check, ruff format --check</code>	1	Any formatting drift	06
G-02	<code>cargo clippy --D warnings, eslint --max-warnings 0, ruff, mypy --strict</code>	1	Any lint	06
G-03	Banned patterns grep	1	<code>unwrap,</code> <code>expect,</code> <code>panic!, todo!,</code> empty <code>catch,</code> bare <code>except,</code> unbounded channel	11, 15
G-04	<code>unsafe</code> audit	1	<code>unsafe</code> outside <code>-sys</code> crates, or missing <code>SAFETY:</code> comment	16
G-05	<code>contract-freeze</code>	1	Frozen artefact changed without an ADR in the same PR	13
G-06	Error registry codegen check	1	Generated bindings differ from <code>errors.toml</code>	11
G-07	Secret scan	1	Any key-shaped string in code, fixtures or test data	19
G-08	<code>cargo deny</code> (licences,	1	Vulnerable or incompatibly	09

#	Gate	Stage	Fails when	Source doc
	advisories, duplicate deps)		licensed dependency	
G-09	Unit tests, all languages	2	Any failure; coverage below 80 % on core crates	05
G-10	Migration chain + <code>PRAGMA</code> <code>integrity_check</code>	2	Any historical catalog fails to migrate	13
G-11	Recipe round-trip with unknown fields	2	Lossy read/write	13
G-12	Per-stage idempotency	2	Double-run snapshots differ	15
G-13	Resume test (kill at 3 points, small fixture)	2	Non-identical gallery or corrupt catalog	15
G-14	Redaction poisoned-fixture test	2	Any PII or key reaches any sink	19
G-15	<code>golden-</code> <code>determinism</code> (DO on Linux CPU)	3	Hash mismatch vs manifest	12
G-16	Cross-EP parity (D1)	3	ΔE or decision- flip beyond tolerance	12
G-17	<code>model-gate</code> metrics + slices + fairness	3	Any slice regresses, or fairness spread widens	18
G-18	Calibration ECE	3	ECE above 0.03	18
G-19	Identity- preservation test	3	Landmark drift or skin-hue shift beyond zero-	14 (F-24)

#	Gate	Stage	Fails when	Source doc
			tolerance bounds	
G-20	Budget assertions on the 300-file mini wedding	4	Any ceiling exceeded; regression above 7 % / 3 %	17
G-21	Platform matrix build + smoke (Win, macOS arm64+x64, Linux)	4	Build or smoke failure on any target	01
G-22	Installer sign + notarise verification on a clean VM	6	Unsigned, un-notarised, or SmartScreen-blocked	30

3. Nightly jobs (non-blocking for PRs, blocking for release)

Job	What it runs	Exit condition
<code>chaos</code>	Six scenarios from <code>14-FMEA-FAILURE-CATALOG.md</code> §8	Zero crashes, zero corruption, accurate quarantine report
<code>fuzz</code>	30 min <code>cargo fuzz</code> on the RAW decoder	Zero new uncontained crashes
<code>full-wedding</code>	3,000-file synthetic wedding end to end on T-Typ and T-Min	Completes inside wall-clock and memory budgets
<code>bench</code>	Dedicated hardware, warm-up + 5 repeats	Updates the rolling baseline; alarms on drift
<code>tsan</code> / <code>loom</code>	Race and interleaving checks	Clean
<code>long-soak</code>	8 hours of repeated runs	RSS flat, zero handle leaks, zero orphan processes

4. Flake policy

Flaky tests are treated as outages, because a flaky suite teaches everyone to press retry — and that is how a real failure ships.

1. A test that fails then passes on retry is auto-labelled `flaky` and reported.
2. Three flakes in seven days → the test is **quarantined into a separate non-blocking job** and an issue is opened with a named owner.
3. A quarantined test must be fixed or deleted within 10 days. Nothing sits in quarantine indefinitely.
4. **Retries are never silent.** A green tick after a retry shows a flake badge.
5. Target flake rate below 0.5 % of runs; the rate itself is a tracked metric.

5. Caching and speed

- `sccache` for Rust, pnpm store cache, model artefacts cached by SHA — never re-downloaded.
- Golden corpus and fixture weddings pulled from LFS with a content-hash cache key.
- Test sharding by historical duration to keep the matrix balanced.
- Merge queue enabled: gates run against the post-merge state, so `main` cannot be broken by two independently-green PRs.

6. Branch protection

- `main` protected. No direct pushes, no force pushes, linear history.
- Required: all stage 1, 2 and 4 gates; stage 3 required when the PR touches `models/`, `shaders/`, `crates/aura-colour/`, or `crates/aura-render/`.
- `CODEOWNERS` review required for every path listed in `contracts.lock`.
- Release tags require the full nightly suite green within the previous 24 hours.

7. Release checklist (stage 6)

- ☐ All nightly jobs green in the last 24 h.
- ☐ Golden manifest unchanged, or changed with `QA1Q` + `COL` sign-off attached.

- ☐ Model set pinned; every model has all five artefacts.
- ☐ Blind A/B panel results recorded for this release.
- ☐ Migrations tested from every released schema version.
- ☐ Installers signed and notarised; verified on clean VMs.
- ☐ Symbols uploaded; crash reporter smoke-tested against this exact build.
- ☐ Rollback artefact from the previous release confirmed installable.
- ☐ Quality changelog written, including anything that changes how galleries look.

8. Acceptance criteria

- ☐ All 22 gates exist as named jobs and are wired to branch protection.
- ☐ Stage 1 completes in under 5 minutes.
- ☐ Flake dashboard live; quarantine process documented and used.
- ☐ Merge queue enabled on `main`.
- ☐ A deliberately broken PR is rejected by the correct gate for each of the 22 — proven once and recorded.

9. Claude Code prompt

Act as `DEVOPS`. Implement the full pipeline in `20-CI-PIPELINE-SPEC.md` as GitHub Actions workflows: `ci-fast.yml`, `ci-correctness.yml`, `ci-quality.yml`, `ci-platform.yml`, `nightly.yml`, `release.yml`. Every gate must be a separately named job so branch protection can require it individually. Then prove the pipeline: for each of the 22 gates, create a throwaway branch that violates exactly that rule and show the specific gate failing. Report a table of gate ID → violating branch → observed failure. Do not make any gate `continue-on-error` to get a green run.